# ECS 160 – Memory ctd.

Instructor: Tapti Palit

Teaching Assistant: **Xingming Xu**

# Reviewing Midterm

Deadline for submissions is Friday, Nov 21 @ 11:59PM

# Agenda

- **Stack canaries**
- Secure programming
- Memory leaks

# Stack Canaries

- Recall that buffer overflows are on the stack
- Mine canaries used to be a warning system
  - Stack canaries do the same for stack overflow
- “Low-level” defense
- Available on most C/C++ compilers

# Using the Stack Canary

- Place random value, the *canary*, onto the stack
  - Right **next to the return address**
- If buffer overflow overwrites the canary, we have detected an overflow
- Compiler recognizes that we shouldn't move on to the real return address, and we exit with __stack_chk_fail() instead of returning to failed code

# Stack Canaries in Java

- Java mitigates all of these issues that require a stack canary
  - Already checks array bounds
  - Already handles type safety and poor casting
  - Etc.
- Still matters when using JNI/native libraries
- JVM is written in C++

# Agenda

- Stack canaries
- **Secure programming**
- Memory leaks

# Recall: Memory Safety

- Memory safety – the ability of a programming language to ensure certain types of memory-related bugs/vulnerabilities don't exist
- Spatial bugs e.g. buffer overflows
- Temporal bugs e.g. pointer issues

# Secure Programming

- Still plenty of ways to *write* insecure code
  - Input validation
  - Poor deserialization etc.
- Programming **patterns** lead to most vulnerabilities

# String Functions

- Strings end in ‘\0’
- Use size-checked variants of common string functions
  - Guarantees that we don’t write outside of expected string memory “domain”

# Temporal Safety Bugs

- Avoid use-after free bugs, by setting freed pointers to 0

```
char* p = malloc(...); // some code follows
free(p);
p = (char*) 0;
```

- C++ also adds standard library (std) referenced pointers
  - std::unique_ptr – once pointer leaves *scope*, **heap** object dropped
  - std::shared_ptr – once *reference count goes to 0*, **heap** object dropped

# C++ Smart Pointers

- std::unique_ptr, std::shared_ptrs are the **modern** (note: late 1990s) way to manage memory
- Goal: eliminate
  - Use-after-free bugs
  - Double-free bugs

# unique_ptr

- **Scope**: region of a code where a variable or function is accessible
- Given a heap-allocated/dynamically allocated object
  - A unique_ptr *owns and manages* that object
- Ownership – when the object goes **out of scope**, unique_ptr deletes it
  - Solves **double-free** bugs!
- Ownership can be moved, at which point the source pointer at the original pointer is set to 0
  - Solves **use-after-free** bugs!

# shared_ptr

- Shared pointers to an object jointly “own” that object
- Maintains a **reference count** of how many pointers there are to that object
- We delete that object when reference count hits 0
  - As such, you can’t ever free an object early
- Only use this when you actually need multiple ownership of objects (which is often)
- Cycles drive the reference counter to deadlock, addressed with weak_ptr

# Agenda

- Stack canaries
- Secure programming
- **Memory leaks**

# Memory Leaks

- Pointers go out of scope, while memory pointing to not freed
- This means space we *think* should be free, is considered by the compiler to be allocated

# Memory Leaks in C/C++

- Typically a result of poor manual memory management
- Non-exhaustive list of causes:
  - Forgetting to delete memory
    - int *x = new int(3); // followed by never deleting x
  - Using deleting[] incorrectly
    - int *x = new int[3]; // must use delete[] x, not regular delete
  - Overwriting a pointer
    - int *x = new int(3);
    - x = new int(5); // original pointer now lost
    - delete x;
  - Throwing an exception before deleting

# Briefly Back To Smart Pointers

- Using smart pointers eliminates many of those traditional memory issues in C++, please use them
- RAII (Resource Acquisition is Initialization) idiom
  - Tying a resource's lifetime to an object's lifetime
  - Acquire resource in constructor, release that resource in destructor

# Memory in Java

- Doesn’t Java have garbage collection?
- Yes, but a memory leak happens when we keep a reference to objects we don’t need anymore
  - Which is a logical issue
- …Preventing the garbage collector from actually cleaning that up

# Garbage Collection

- Java Runtime needs to keep track of all dynamically allocated objects somehow
- Solution: reachability graph
- **Garbage**: objects that the *program* cannot reach or use
  - Garbage collector threads collect objects that the program cannot get

# Algorithms

- JVM provides a bunch, the one we consider is Mark and Sweep
- Mark and Sweep
  - Track all objects from root node
  - Language runtime spawns garbage collector threads upon **reaching memory threshold**
  - **Mark**: from root, identify objects no longer reachable or usable
  - **Sweep**: delete marked objects
- NOTE: this happens when we reach around that threshold
- This is clearly a costly operation

# Mark and Sweep

- This is a graph traversal problem
- Basic Implementation:
  - Iterative DFS (to prevent stack overflow) w/ marked bits, originally set to 0
  - Identify all reachable bits with DFS, set those bits to 1
  - Clear memory for any items with bits set to 0
- Handles cycles beautifully, little additional overhead
- Can potentially cause heap fragmentation
  - Heap ends up with many small, non-contiguous, free regions, which is unusable for large chunks

# Done!